

# Nepho character art — provider brief

What to choose, how to connect it, how spending is capped, and how to prove which model draws better. Facts below are from the Gemini API and OpenAI docs as read on 13 Sep 2026; the pipeline that implements them is `tools/art-pipeline.mjs` (`npm run art:queue`).

## 1 • What to choose

Both providers changed under us this year. The relevant current models, at the standard's target (a 3×3 sheet, 2048×2048, transparent):

|                                           | OpenAI • gpt-image-2.5-sunburst                                                   | Gemini • gemini-3-pro-image (Nano Banana Pro)                             | Gemini • gemini-3.1-flash-image                                            |
|-------------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Native 2048 <sup>2</sup> output           | Yes — custom WIDTHxHEIGHT , any size up to 4K pixels                              | Yes — <code>imageSize: "2K"</code>                                        | Yes — 2K (4K available)                                                    |
| Real alpha channel                        | Yes — <code>background: "transparent"</code> , PNG                                | No — flat magenta matte, intake keys it                                   | No — same                                                                  |
| Reference images                          | Several via the edits endpoint (we send 3)                                        | Up to 14: 5 character + 3 style + 6 object                                | Up to 14: 4 character + 10 object                                          |
| Instruction following                     | "Editing precision" model; docs warn on layout-sensitive composition              | Built-in "thinking" pass before rendering — made for complex instructions | Minimal thinking by default ( <code>thinking_level: high</code> available) |
| Cost per action file (2048 <sup>2</sup> ) | <b>\$0.107</b> high • \$0.19 xhigh • \$0.43 max                                   | <b>\$0.134</b> (1K–2K same price)                                         | <b>\$0.101</b>                                                             |
| Cost per character sheet (2:1)            | \$0.04–0.07                                                                       | \$0.134                                                                   | \$0.10                                                                     |
| Proven on our gate                        | Yes — the gpt-image family passed punk, ferryman, shmuel, glass-warden, kilnheart | Not yet                                                                   | Not yet                                                                    |
| Free tier for images                      | None (org verification may be required)                                           | None — Free tier returns 429                                              | None                                                                       |

**Recommendation.** Default to **Sunburst at high , 2048<sup>2</sup>, transparent** for action files — the only option that delivers the standard's exact container with no keying, at the lowest proven cost. Use **Gemini 3 Pro Image** as the alternate provider: the pipeline hands a file that failed three times to it automatically, and it is the natural choice for *character sheets* (thinking + 14 references). Decide the default per rank with the benchmarker (\$4) on one boss and one enemy before committing the queue to either.

Both docs list a batch mode (Gemini: 50 % off, ≤ 24 h turnaround) — right for a later bulk re-pass, wrong for a same-day queue.

## 2 • How to integrate and connect

## Gemini (AI Studio API)

1. aistudio.google.com → **API keys**. The existing key ( AQ.... ) already reaches `generativelanguage.googleapis.com`; text models answer, image models are blocked only by tier.
2. On the project's row click **Set up billing**. New accounts land on the **Prepay** plan: link a billing account and buy credits (minimum \$5). Tier 1 is granted instantly.
3. Do **not** enable auto-reload (see §3).
4. Optional: `GEMINI_IMAGE_MODEL=gemini-3.1-flash-image` to use Nano Banana 2 instead of Pro.

## Connect to the repo

```
cat > .env.local <<'EOF'          # gitignored; read by the pipeline, never logged
OPENAI_API_KEY=sk-proj-...
GEMINI_API_KEY=AQ....
EOF
node tools/art-pipeline.mjs --smoke --provider gpt          # one 20482 image + one judge call
- proves key, model, size, alpha
node tools/art-pipeline.mjs --smoke --provider gemini
npm run art:queue          # the queue in
docs/prompts/README.md, unattended
```

The pipeline reads `.env.local` then the environment; it picks `gpt` when an OpenAI key exists, else `gemini` (`--provider` overrides). Requests are assembled verbatim from `docs/prompts/<id>.md`; every file goes through `intake:character` → `verify:character` → the vision judge (Gemini `gemini-flash-latest`, fractions of a cent) → retry with the failing rule bolded; a passing character is built, tested and committed on its own.

## OpenAI

1. platform.openai.com → **Settings** → **Organization** → **Verification** if prompted (GPT Image models may require it).
2. **Projects** → **Create** "nepho-art". In the project: **Limits** → monthly budget \$50, *block requests* at the limit; allowed models: `gpt-image-2.5-sunburst`, `gpt-image-2.5-flare`.
3. **Billing** → buy \$50 of prepaid credits; auto-recharge **off**.
4. **API keys** → create a key scoped to that project.

### 3 · Safe spending — four layers, each independent

| Layer                   | Where                                                                                                                          | What it stops                                                                                                                                                                                                                                                                                                                         |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.<br>Prepaid credits   | Gemini Prepay plan · OpenAI prepaid balance, auto-recharge off                                                                 | Everything. Gemini: "when your Prepay credit balance hits \$0, all API keys in all projects linked to that billing account stop working" (≈10 min billing latency). Buy exactly the amount you are willing to lose.                                                                                                                   |
| 2.<br>Provider budget   | OpenAI project monthly budget with <i>block</i> · Gemini Tier 1 spend-rate limit (\$10 per rolling 10 min, enforced by Google) | A runaway loop: rejected with 429 before the balance drains.                                                                                                                                                                                                                                                                          |
| 3.<br>Pipeline hard cap | <code>--budget</code> (default \$60 per run) and <code>--char-budget</code> (default \$8 per character)                        | The script estimates every request from the docs' price table <i>before</i> sending it and refuses any that would cross the cap; OpenAI responses settle the ledger with real token usage. Spend is persisted in <code>public/assets/backups/spend.json</code> across restarts and printed per request in <code>QUEUE-LOG.md</code> . |
| 4.<br>Throttles         | <code>GEMINI_CONCURRENCY=6</code> , <code>OPENAI_CONCURRENCY=8</code>                                                          | Bursts that would trip the \$10/10 min limit or provider rate limits (429 storms cost retries, not money, but they cost time).                                                                                                                                                                                                        |

#### Forecast for the remaining queue (14 characters, 146 requests)

| Scenario                                           | Requests | Sunburst high | Gemini Pro | Sunburst xhigh (heroes only) |
|----------------------------------------------------|----------|---------------|------------|------------------------------|
| Clean run                                          | 146      | ≈ \$15        | ≈ \$20     | + \$2.50                     |
| With 50 % retries (Codex's observed rate is ~35 %) | ≈ 220    | ≈ \$23        | ≈ \$29     | + \$4                        |
| Judge calls (≈ 2 per file)                         | ≈ 440    | ≈ \$1 total   |            |                              |

Either provider finishes the queue inside **\$25–35**. \$50 of prepaid credit per provider is enough with margin; `--budget 60` is never reached on a healthy run and is the backstop if something loops.

### 4 · Better results — the benchmarker

```
node tools/art-pipeline.mjs --bench monk-zero      # a boss
node tools/art-pipeline.mjs --bench brawler       # an enemy
```

For one character it generates, on **every provider that has a key**, the character sheet plus the two files that fail most ( `idle` and `walk / approach` ), through the real intake → gate → judge → retry loop, then appends a table to `docs/prompts/BENCH.md` :

| provider | model                       | sheet     | idle              | approach           | spend  | model time |
|----------|-----------------------------|-----------|-------------------|--------------------|--------|------------|
| gpt      | gpt-image-2.5-sunburst high | ok (41 s) | PASS in 1 attempt | PASS in 2 attempts | \$0.38 | 190 s      |
| gemini   | gemini-3-pro-image          | ok (58 s) | PASS in 1 attempt | PASS in 1 attempt  | \$0.41 | 205 s      |

Illustrative rows — the real table is written by the run. The generated files are kept under `public/assets/backups/bench/<id>/<provider>/` for a side-by-side look; nothing bench-made stays under `actions/`.

**Decision rule:** fewer attempts to PASS wins (each retry is a whole image); on a tie, the judge's identity verdict, then cost. Run it once per rank (bosses and enemies differ: 6 vs 10 files, bigger figures) — about \$1 per bench — then set `--provider` for the queue. Quality levers beyond model choice, all already wired: the accepted `idle.png` attached to every later file (size lock), the sheet attached to every request (identity lock), the gate's own wording bolded on retries, and `--quality xhigh` for the two heroes whose faces carry a player photo.

## 5 • Today

---

1. Billing on both providers (10 minutes). `--smoke` on each.
2. `--bench monk-zero` and `--bench brawler` ( $\approx 15$  min,  $\approx \$2$ ). Pick the default provider per rank.
3. Stop the Codex thread (its files are kept: the pipeline resumes any partial set from its sheet).
4. `npm run art:queue -- --provider <winner>` — 3 characters in parallel,  $\approx 45$ –70 minutes for the 14 remaining, each committed as it passes. Watch `docs/prompts/QUEUE-LOG.md`; the run ends with a notification and a spend total.
5. Place the new characters on levels in `/backoffice.html`; watch each once on `/showcase.html`.

Sources: [ai.google.dev/gemini-api/docs](https://ai.google.dev/gemini-api/docs) — image-generation, interactions, rate-limits, billing, batch-api, pricing · [platform.openai.com/docs](https://platform.openai.com/docs) — guides/image-generation (incl. the output-token calculator), pricing.